

Spec Kit — Complete Guide

Code-First Approach — GHCP Rollout Dashboard

From Start to End

Approach: Code-first — no BRD, no FRD

Manager requirement: Point AI at code, derive spec from code

Goal: E2E tests + second feature with updated tests

BEFORE YOU START — One Time Setup

Run this in your project root terminal:

```
powershell

# Navigate to project root
cd "C:\EA Work\Spec-kit\Rollout-dashboard-using-speckit"

# Verify Spec Kit is initialised
ls .specify
ls .github\prompts

# Verify Playwright installed
pip list | findstr playwright

# If playwright missing
pip install pytest-playwright
python -m playwright install chromium

# Verify pytest-html installed
pip list | findstr pytest-html

# If pytest-html missing
pip install pytest-html
```

COMMAND 1 — Constitution

Open Copilot Agent in VS Code → paste this

```
/speckit.constitution
```

```
Testing framework: pytest-playwright, single test file only  
Security: parse HTML as string for security checks  
Simplicity: plain pytest functions, no helper classes  
Immutability: never modify existing generation scripts  
Output: tests/screenshots/rollout/ and tests/reports/
```

Wait for: constitution.md created

Then: Click Continue when asked

COMMAND 2 — Specify (code-first — no BRD)

Paste this — attach nothing

```
/speckit.specify
```

```
Read the source code at Package/Rollout/ including  
build_rollout_dashboard.py and rllot_dashboard.html  
  
Understand what this dashboard does from the code itself.  
Generate a spec for adding Playwright E2E tests for it.  
Do not ask me for requirements – derive from code only.
```

Wait for: spec.md created with FR-XXX from code

Then: Answer any [NEEDS CLARIFICATION] questions

COMMAND 3 — Plan

Paste this

```
/speckit.plan
```

```
Tech stack:  
- Playwright Python with pytest-playwright
```

- Single test file: tests/test_rollout_dashboard.py
- HTML target: Package/Rollout/rllot_dashboard.html
- JSON source: Package/Rollout/rllot_data.json
- Screenshots: tests/screenshots/rollout/baseline.png
- Reports: tests/reports/rollout_report.html

Do not modify: config.py, process_rollout.py,
build_rollout_dashboard.py

Test order: structure → data accuracy → security → screenshot

Validate against constitution. Generate plan.md,
research.md, quickstart.md.

Wait for: plan.md, research.md, quickstart.md created

Then: Verify constitution gates all show PASS

COMMAND 4 — Analyze (do not skip)

Paste this

```
/speckit.analyze
```

```
Read spec.md, plan.md, constitution.md for feature  
001-rollout-dashboard-e2e-tests.
```

```
Find any gaps between spec and plan.  
Find any constitution violations.  
Find any FR-XXX not addressed in plan.  
Report only – do not modify any files.
```

Wait for: Analysis report

Then: Fix any WARNING or FAIL items before tasks

COMMAND 5 — Tasks

Paste this

```
/speckit.tasks
```

```
Generate tasks for 001-rollout-dashboard-e2e-tests.
```

Phase order:

Phase 1: Setup – verify files, verify config entry

Phase 2: Structure tests – page load, title, sections

Phase 3: Data accuracy tests – HTML vs JSON

Phase 4: Security tests – CSP, no inline handlers

Phase 5: Screenshot baseline

Phase 6: Documentation – code path for manager

Each task must show: file path, function name, FR-XXX covered.

Tests only – no implementation tasks.

Wait for: tasks.md created

Then: Confirm phase order is correct

COMMAND 6 — Implement

Paste this

```
/speckit.implement
```

```
Execute tasks for 001-rollout-dashboard-e2e-tests.
```

Rules:

- Read constitution.md first
- Single file: tests/test_rollout_dashboard.py
- Each test must have comment: # Validates: FR-XXX
- Data tests read JSON dynamically – no hardcoded values
- Security tests parse HTML as raw string
- Do not modify existing scripts
- Show complete test file before marking tasks done

Wait for: tests/test_rollout_dashboard.py shown

Review the file — check:

RUN TESTS

In terminal

Expected: 7 passed, 0 failed

Then: Open tests/reports/rollout_report.html in Chrome

COMMAND 7 — Checklist (manager sign-off)

Paste this

Format for non-technical manager.

Output: checklists/rollout.md — share with manager

ITERATION 2 — Add New Feature

After iteration 1 is complete and tests pass

This is what your manager specifically asked for.

Pick any small new functionality to add to the dashboard.

Example new feature ideas:

- Add a "Last Updated By" field to dashboard
- Add a summary count row at bottom of each section
- Add a colour indicator for completion status
- Add a filter to show only incomplete teams

Paste this for new feature

```
/speckit.specify
```

```
Read Package/Rollout/build_rollout_dashboard.py  
and Package/Rollout/rllot_dashboard.html
```

```
I want to add [YOUR NEW FEATURE] to this dashboard.
```

```
Generate a spec for:
```

1. The new feature implementation
2. E2E test updates for this new scenario

```
Both must be in the same spec and same plan.
```

```
Derive existing context from code – do not ask for BRD.
```

Then follow same commands 3-7 again for feature 002

When running implement for feature 002

```
/speckit.implement
```

```
Execute tasks for 002-[new-feature-name].
```

Important: tests/test_rollout_dashboard.py already exists.
Add new test function for the new feature – do not
overwrite existing tests.
Existing 7 tests must still pass after new feature added.

TRACEABILITY REVIEW — How to validate spec → code

This is what your manager asked: "how do you review
without reading every line of code?"

The FR-XXX chain check

After implement completes — run this mental check:

```
Step 1: Open spec.md
        Count all FR-XXX numbers
        Example: FR-001 to FR-007

Step 2: Open plan.md
        Every FR-XXX from spec must appear in plan
        If FR-004 missing from plan → gap found

Step 3: Open tasks.md
        Every FR-XXX must have a task assigned to it
        If FR-005 has no task → not implemented

Step 4: Open test_rollout_dashboard.py
        Every test must have # Validates: FR-XXX comment
        If FR-006 has no test function → missing

Step 5: Open checklists/rollout.md
        Every FR-XXX must have a checkbox
        This is your sign-off gate
```

Quick grep check (terminal)

```
powershell
```

```
# Check FR-004 appears in all artifacts
findstr "FR-004" .specify\specs\001-rollout-dashboard-e2e-tests\spec.md
findstr "FR-004" .specify\specs\001-rollout-dashboard-e2e-tests\plan.md
findstr "FR-004" .specify\specs\001-rollout-dashboard-e2e-tests\tasks.md
findstr "FR-004" tests\test_rollout_dashboard.py
findstr "FR-004" .specify\specs\001-rollout-dashboard-e2e-tests\checklists\rollout.m
```

If FR-004 appears in all 5 files → fully traced.

If missing from any file → gap in chain → fix before sign-off.

LESSONS LEARNED — Document these for PPT

Lesson 1 — Agent mode vs command-by-command

Problem: Running constitution prompt in Agent mode caused AI to skip all subsequent commands and implement everything in one shot

Learning: Run each /speckit.* command separately
Review output before running next command
Agent mode = autonomous, no human checkpoints

Lesson 2 — Skipping plan.md breaks the chain

Problem: Running /speckit.tasks without plan.md caused tasks to have wrong file paths and no architecture decisions

Learning: Never skip /speckit.plan
Every command reads previous artifacts
Missing artifact = broken reading chain

Lesson 3 — BRD vs code-first

Problem: Starting with BRD creates two sources of truth
Code says one thing, BRD says another
AI gets confused by conflicting sources

Learning: Point AI directly at existing code

AI reads code and derives spec automatically
One source = no ambiguity

Lesson 4 — Constitution scope matters

Problem: Constitution prompt too detailed (included file paths and test scenarios) caused Agent to treat it as a task and implement directly

Learning: Constitution = rules only, never task descriptions
Keep it to 5-6 lines maximum
Rules: framework, security, simplicity, immutability

Lesson 5 — How to verify without reading every line

Technique: FR-XXX chain tracing
Check each FR appears in:
spec → plan → tasks → code → checklist
If chain complete → requirement implemented
No need to read every line of code

ALL ARTIFACTS — What you end up with

```
.specify/  
  memory/  
    constitution.md          ← project law  
  
  specs/  
    001-rollout-dashboard-e2e-tests/  
      spec.md                ← FR-XXX requirements  
      plan.md                ← architecture decisions  
      research.md            ← tech decisions  
      data-model.md          ← JSON structure  
      tasks.md               ← ordered task list  
      quickstart.md          ← how to run  
      checklists/  
        rollout.md           ← manager sign-off  
  
    002-[new-feature]/       ← iteration 2  
      spec.md
```

plan.md	
tasks.md	
checklists/	
tests/	
test_rollout_dashboard.py	← all E2E tests
screenshots/	
rollout/	
baseline.png	← visual baseline
reports/	
rollout_report.html	← test report
Package/	
Rollout/	
rllot_dashboard.html	← dashboard (unchanged)
rllot_data.json	← data (unchanged)

COMMAND SUMMARY — Quick reference

Step	Command	Output	Skip?
1	/speckit.constitution	constitution.md	Never
2	/speckit.specify	spec.md	Never
3	/speckit.plan	plan.md + research	Never
4	/speckit.analyze	analysis report	Never
5	/speckit.tasks	tasks.md	Never
6	/speckit.implement	test code	Never
-	pytest	test report	Never
7	/speckit.checklist	sign-off doc	Never

FOR MANAGER PPT — Key points to cover

Slide 1: Problem we solved

- Manual dashboard validation was time consuming
- No automated regression after OWASP fixes
- No proof security fixes held after regeneration

Slide 2: Approach – Code-first SDD

- No BRD required
- AI reads existing code
- Spec derived from code automatically
- One source of truth – no ambiguity

Slide 3: What Spec Kit generated

- spec.md with FR-XXX from code
- plan.md with architecture decisions
- tasks.md with ordered implementation
- 7 passing Playwright tests
- HTML test report
- Manager sign-off checklist

Slide 4: Traceability technique

- FR-XXX chain: spec → plan → tasks → code → checklist
- grep check proves every requirement implemented
- No need to read every line of code

Slide 5: Lessons learned

- Agent vs command-by-command
- Never skip plan.md
- Constitution = rules only
- Code-first beats BRD-first

Slide 6: Second iteration – new feature

- New feature added to dashboard
- E2E tests updated in same plan
- Proves the process is repeatable